



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Organización y Arquitectura de las Computadoras.

Práctica No. 5: Implementación de Algoritmos de Procesamiento de Cadenas mediante Direccionamiento de Memoria.

Alumno:

Joshua Osorio O. – 1293271

Docente:

José Isabel García R.

Marzo/2026

Tabla de contenido

Objetivo	2
Desarrollo	2
Conclusiones.....	12
Dificultades	10
Bibliografía	12

Objetivo

Seleccionar el modo de direccionamiento adecuado, para manejo de memoria en aplicaciones de sistemas basados en microprocesador, mediante el análisis de los diversos modos de direccionamiento existentes, de forma responsable y eficiente.

Desarrollo

Parte I (25%)

Implementar una subrutina la cual debe buscar, contar y mostrar en que posición de una cadena de texto aparece una letra ingresada por el usuario, recibe en edx la dirección de la cadena a utilizar, y dentro de la subrutina debe pedir la letra a buscar.

Ejemplo:

```
mv edx, cadena ;la cadena es 'hola mundo'
call proc1 ;buscaremos la letra o
imprime en terminal lo siguiente:
    1 ;la letra se encontró en posición 1.
    9 ;la letra se encontró en posición 9.
    2 ;la letra se encontró 2 veces.
```

UABC_FCQI_ORGANIZACIÓN_Y_ARQUITECTURA_DE_COMPUTADORAS.

```

buscarLetra:
;  Entrada:  edx: direccion de cadena, edi: longitud cadena
;  Utiliza:
;  Salida:
mov     esi,     0
mov     ecx,     edx    ; Guardar temporalmente dir
mov     ebx,     0
mov     edx,     msj1
call    puts
call    getche
mov     bl,      al    ; Caracter a buscar en bl
call    salto
.ciclo_buscar:
    cmp     esi,     edi
    je      .fin_buscar
    push    ebx
    mov     bh,      [ecx + esi]
    cmp     bh,      bl    ;Letra capturaeda vs letra cadena
    pop     ebx
    je      .encontrada
    inc     esi
    jmp     .ciclo_buscar
.encontrada:
    mov     edx,     msj2
    call    puts
    mov     eax,     esi    ; cargar numero
    push    esi
    mov     esi,     temp    ; Cargar cadena
    call    printHex
    pop     esi
    call    salto
    inc     bh
    inc     esi
    jmp     .ciclo_buscar
.fin_buscar:
    push    esi
    mov     edx,     msj3
    call    puts
    mov     eax,     0
    mov     al,      bh
    mov     esi,     temp
    call    printHex
    pop     esi
    mov     edx,     msj33
    call    puts
ret

```

```

mov     edx,      pasol
call    new_puts
mov     edx,      msj0
call    new_puts
mov     edx,      cadena
mov     ecx,      254    ; Se indica la longitud de cadena.
call    inputStr
call    buscarLetra

```

Practica 5

Parte 1 - contar letras

Ingresa cadena: hola mundo soy joshua

Ingresa la letra a buscar: o

La letra se encontro en la posicion 00000001

La letra se encontro en la posicion 00000009

La letra se encontro en la posicion 0000000C

La letra se encontro en la posicion 00000010

La letra se encontro 00000004 veces

Parte II (25%)

Implementar una subrutina donde se reciben dos cadenas, la primera está direccionada por edx y tiene la cadena de origen, la segunda está direccionada por edi y tiene la cadena de destino, se debe de tomar la cadena origen e invertirla y guardarla en la cadena destino, al final se debe mostrar en terminal.

```

mov     edx,      paso2    ; Mostrar mensaje.
call    new_puts
mov     edx,      cadena
mov     edi,      cadena2
call    invertir
mov     edx,      msj4
call    new_puts
mov     edx,      cadena
call    new_puts
call    salto
mov     edx,      msj5
call    new_puts
mov     edx,      cadena2
call    new_puts
call    salto
    
```

```
invertir:
; Entrada:  edx = dir origen
;          edi = dir destino
; Utiliza:
; Salida:
; Nota: La cadena oringen debe finalizar con caracter nulo
push     edx          ; resguardar direccion origen
push     edi          ; Resguardar direccion destino
mov      ecx, 0       ; Limpiar registro.
mov      esi, 0       ; Limpiar registro.
.ciclo_push:
    mov     cl, [edx]
    cmp     cl, 0
    je      .ciclo_pop
    inc     edx
    inc     esi
    push    ecx
    jmp     .ciclo_push
.ciclo_pop:
    cmp     esi, 0
    je      .fin_invert
    pop     ecx
    mov     byte[edi], cl
    inc     edi
    dec     esi
    jmp     .ciclo_pop
.fin_invert:
    inc     edi
    mov     byte[edi], 0
    pop     edx
    pop     edi
ret
```

En terminal se debera de mostrar lo siguiente.

```
Paso 2 - invertir cadena
Cadena original: hola mundo soy joshua
Cadena invertidaL: auhsoj yos odnum aloh
```

Parte III (25%)

Implementar una subrutina que reciba una cadena de texto direccionada por edx y la pase a mayúsculas, al final se debe mostrar en terminar con new_puts.

```

upperCase:
; Entrada:  edx = dir cadena
; Salidan:  Cadena en mayusculas.
push edx
push eax
.ciclo_upperCase:
mov     al, [edx]
cmp     al, 0
je      .fin_upperCase
cmp     al, 32 ; comparar con espacio
je      .espacio
; call   upperLater
sub     al, 32
.espacio:
mov     byte[edx], al ; guardar letra casteada
inc     edx ; moverme a la siguiente posicion
jmp     .ciclo_upperCase
.fin_upperCase:
pop     edx
pop     eax
ret

mov     edx, paso3
call    new_puts
mov     edx, P3msj0
call    new_puts
mov     edx, cadena
call    new_puts
mov     edx, cadena
call    salto
call    upperCase
mov     edx, P3msj1
call    new_puts
mov     edx, cadena
call    new_puts
call    salto

```

En terminal se debera de mostrar lo siguiente.

```

Paso 3 - Uper casting
Cadena original: hola mundo soy joshua
Cadena en mayusculas: HOLA MUNDO SOY JOSHUA

```

Parte IV (25%)

Implementar una subrutina que reciba una cadena de texto direccionada por edx, debe contar cuantas vocales aparecen en la cadena, debe mostrar la cantidad individual y total de ellas.

```

mov     edx, paso4
call    new_puts
mov     edx, p4msj
call    new_puts
mov     edx, cadena
call    new_puts
call    salto
mov     edx, cadena
call    contar_vocales
call    salto

```

UABC_FCQI_ORGANIZACIÓN_Y_ARQUITECTURA_DE_COMPUTADORAS.

```
contar_vocales:
; Entrada: edx = Dir de la cadena a evaluar.
; Tener una 5 variables de 1 byte para contar
push    edx
push    ecx
push    eax
push    edi
```

```
mov     edi, 0
mov     ecx, 0
```

```
.ciclo_recorrer:
```

```
mov     al, [edx + edi]
cmp     al, 0
je      .fin_recorrer
call    vocal
inc     edi
jmp     .ciclo_recorrer
```

```
.fin_recorrer:
```

```
cmp     ch, 0
je      .no_Vocales
```

```
.si_vocales:
```

```
mov     edi, 0
call    putchar
```

```
.ciclo_datos:
```

```
mov     edx,    Caracter_voacales
mov     al, [edx + edi]
cmp     al, 0
je      .fin_contar
call    putchar
mov     edx, p4msj1
call    puts
call    mostrar_cantidad_letra
inc     edi
call    salto
jmp     .ciclo_datos
```

```
.no_Vocales:
```

```
mov     edx, p4msj0
call    puts
```

```
.fin_contar:
```

```
pop     edi
pop     eax
pop     ecx
pop     edx
```

```
ret
```

```

vocal:
; Entrada:  al = caracter en codigo ASCII.
; Salida:   cl = 0/1 para saber si es vocal.
push     edx
mov      cl, 0
; Comparar vocales minisculas.
cmp      al, 'a'
je       .es_a
cmp      al, 'e'
je       .es_e
cmp      al, 'i'
je       .es_i
cmp      al, 'o'
je       .es_o
cmp      al, 'u'
je       .es_u
; Comparar vocales mayúsculas.
cmp      al, 'A'
je       .es_a
cmp      al, 'E'
je       .es_e
cmp      al, 'I'
je       .es_i
cmp      al, 'O'
je       .es_o
cmp      al, 'U'
je       .es_u
; Si no hacer nada.
jmp      .fin_vocal

.es_a:
mov      edx, vocal_A
add      byte[edx], 1
jmp      .es_vocal
.es_e:
inc      byte[vocal_E]
jmp      .es_vocal
.es_i:
inc      byte[vocal_I]
jmp      .es_vocal
.es_o:
inc      byte[vocal_O]
jmp      .es_vocal
.es_u:
inc      byte[vocal_U]
jmp      .es_vocal
.es_vocal:
mov      ch, 1      ; Para mentener el valor de si es vocal.
.fin_vocal:
pop      edx
ret

```



```

mostrar_cantidad_letra:
    push    esi
    push    eax
    push    edx

    mov     esi,     temp ; Se realiza para poder utilizar printHex

    ; Como manejo edi como indice de AEIOU
    ;           edi     01234
    cmp     edi, 0
    je      .count_a

    cmp     edi, 1
    je      .count_e

    cmp     edi, 2
    je      .count_i

    cmp     edi, 3
    je      .count_o

    cmp     edi, 4
    je      .count_u

.count_a:
    mov     edx,     vocal_A
    mov     eax,     [edx]
    call    printHex
    jmp     .fin_count

.count_e:
    mov     edx,     vocal_E
    mov     eax,     [edx]
    call    printHex
    jmp     .fin_count

.count_i:
    mov     edx,     vocal_I
    mov     eax,     [edx]
    call    printHex
    jmp     .fin_count

.count_o:
    mov     edx,     vocal_O
    mov     eax,     [edx]
    call    printHex
    jmp     .fin_count

.count_u:
    mov     edx,     vocal_U
    mov     eax,     [edx]
    call    printHex
    jmp     .fin_count

.fin_count:
    pop     edx
    pop     eax
    pop     esi
ret

```

En terminal se debera de mostrar lo siguiente.

```
Paso 4 - Contrar vocales
Cadena a revisar: HOLA MUNDO SOY JOSHUA
A Se encontraron 00000002
E Se encontraron 00000000
I Se encontraron 00000000
O Se encontraron 00000004
U Se encontraron 00000002
```

Instrucciones vistas en clase y permitidas para esta práctica.

1. add
2. sub
3. mov
4. jmp
5. cmp
6. je
7. loop
8. getch
9. getche
10. puts
11. putchar
12. inc
13. dec
14. new_puts
15. new_gets

Obligatoriamente de cada paso debe tener las capturas del código y de la terminal.

Dificultades

Durante el desarrollo de esta práctica, se presentaron diversos retos técnicos relacionados con la gestión de memoria de bajo nivel:

Gestión de Contadores: Una de las mayores complicaciones fue determinar la estrategia adecuada para el almacenamiento de los contadores. Se decidió declarar variables específicas en la sección .bss para el contador general y para cada una de las cinco vocales (vocal_A a vocal_U), asegurando que los valores no se perdieran al limpiar los registros entre subrutinas.

UABC_FCQI_ORGANIZACIÓN_Y_ARQUITECTURA_DE_COMPUTADORAS.

Manejo de Punteros e Índices: El uso de direccionamiento indirecto ([edx + edi]) requirió un control estricto de los registros de índice para evitar el acceso a posiciones de memoria fuera del rango de las cadenas definidas (254 bytes).

Inversión de Cadena (Pila vs. Registros): La lógica para invertir la cadena implicó un manejo preciso del Stack (Pila). El reto consistió en empujar (push) los caracteres de forma secuencial y extraerlos (pop) correctamente para lograr el orden inverso sin corromper otros registros activos.

Conversión de Caracteres (Casting): En el proceso de "Upper Casting", se debió tener cuidado con el valor ASCII de los espacios (32) y otros caracteres no alfabéticos para evitar restas accidentales que resultaran en caracteres no imprimibles.

Al principio resultó complejo utilizar los registros edi y esi porque nunca se preguntó dónde se inicializaban y se daba por hecho que al momento de utilizar getche se iniciaba además de coordinar los registros esi y edi para recorrer la cadena desde ambos extremos sin perderse, especialmente al momento de detener el ciclo cuando los índices se cruzaban.

Se solucionó utilizando cmp esi, edi y saltos condicionales (je, jne) para decidir cuándo terminar.

Uso del delimitador personalizado

La decisión de usar el carácter '+' como fin de captura, en lugar del tradicional Enter, obligó a modificar tanto inputStr como outputStr para que ambas reconocieran el mismo símbolo.

Fue necesario asegurar que el terminador nulo (0) se agregara después del último carácter capturado, para poder utilizar puts de la librería sin problemas.

Coordinación entre subrutinas

Al principio, inputStr no retornaba la longitud de la cadena, lo que dificultaba su uso en palindromo. Se corrigió almacenando la longitud en edi y manteniendo ese valor para la siguiente rutina.

También hubo que tener cuidado de no modificar registros que contenían valores importantes (como la dirección de la cadena) sin antes preservarlos.

Control de saltos y bucles

En ocasiones los bucles no terminaban correctamente debido a comparaciones mal planteadas o a la omisión de incrementos/decrementos.

Se depuró cada ciclo verificando manualmente los valores de los registros con pequeños mensajes de prueba (aunque no se incluyen en el código final).

Compatibilidad con la librería `pc_io.inc`

Aunque se usaron las funciones `getche`, `putchar` y `puts` de la librería, hubo que entender cómo afectaban los registros (por ejemplo, `puts` modifica `edx`) para no perder datos importantes.

Se optó por guardar en `push/pop` los registros necesarios en las subrutinas críticas.

A pesar de estas dificultades, se logró un programa funcional que cumple con todos los requisitos de la práctica, consolidando así los conocimientos de programación en ensamblador y manejo de

Conclusiones

Dominio de Direcccionamiento: Se logró cumplir el objetivo de seleccionar y aplicar el modo de direccionamiento adecuado. El uso de direccionamiento por registro indirecto resultó ser la forma más eficiente para recorrer y manipular arreglos de caracteres (cadenas).

Modularidad mediante Subrutinas: La implementación de subrutinas (`buscarLetra`, `invertir`, `upperCase`, `contar_vocales`) permitió un código más limpio y escalable, facilitando la depuración individual de cada algoritmo de procesamiento.

Gestión de Memoria: La práctica reafirmó la importancia de la sección `.data` y `.bss` en la arquitectura x86. Entender cómo se reservan los bytes y cómo se cargan sus direcciones en registros como `EDX` o `EDI` es fundamental para el manejo de aplicaciones basadas en microprocesadores.

Optimización de Instrucciones: Al limitarse a un conjunto de instrucciones específico (instrucciones de movimiento, comparación y saltos), se comprendió la lógica fundamental que siguen los lenguajes de alto nivel para realizar tareas complejas de manipulación de texto.

Bibliografía

- [1] D. A. Alpern, "El microprocesador 80386." <https://www.alpertron.com.ar/80386.HTM>
- [2] "Guide to x86 Assembly." <https://www.cs.virginia.edu/~evans/cs216/guides/x86.html>